



Hola Mundo

Python + FastAPI + SQLAlchemy + PostgreSQL + Docker

- **Lenguaje y Frameworks seleccionados:**

Componente	Elección	Motivo
Lenguaje	Python 3.12	Tipado moderno, ecosistema maduro, compatible con FastAPI y SQLAlchemy
Framework	FastAPI 0.115	Alto rendimiento (ASGI), validación automática con Pydantic, docs Swagger integradas
Servidor	Uvicorn	Servidor ASGI recomendado para FastAPI, soporte async nativo
Frontend	React (separado)	El frontend React no forma parte de este tutorial (solo backend)

- **Librerías ORM utilizadas:**

- **SQLAlchemy:** ORM principal. Mapea clases Python a tablas de PostgreSQL. La versión 2.0 introduce la API declarativa moderna con type hints nativos.
- **psycopg2-binary:** Driver que permite a SQLAlchemy comunicarse con PostgreSQL. La variante -binary incluye las dependencias compiladas, facilitando la instalación.
- **python-dotenv:** Carga las variables de entorno desde el archivo .env (URL de conexión, credenciales, etc.) sin exponerlas en el código.

```
requirements.txt
1 fastapi>=0.115,<0.116
2 uvicorn[standard]>=0.30,<0.31
3 sqlalchemy>=2.0,<2.1
4 psycopg2-binary>=2.9,<3.0
5 python-dotenv>=1.0,<2.0
6
```

- **Configuración y levantamiento de la base de datos:**

Requisitos previos:

→ [Docker Desktop](#) instalado y en ejecución.

→ [Python 3.12+](#) instalado.

→ [VS Code](#) con una terminal abierta en la raíz del proyecto ([project-backend](#)).



Descripción general del proceso:

El proceso consiste en:

- Configurar las variables de entorno del proyecto.
- Levantar el contenedor de la base de datos usando Docker.
- Crear y activar un entorno virtual de Python.
- Instalar las dependencias necesarias.
- Ejecutar el backend para conectarse a la base de datos.

Flujo sin automatización (Paso a paso):

Para Windows (PowerShell):

Crear el archivo .env: Primero creamos el archivo para que la aplicación tenga su configuración y credenciales.

- **Comando:** `copy .env.example .env`

Verificación: Abre el archivo **.env** en Visual Studio Code y asegúrate de que el usuario, contraseña, puerto y nombre de base de datos coincidan con el archivo **docker-compose.yml**.

Levantar Base de Datos (Docker): Esto inicia el contenedor de la base de datos.

- **Comando:** `docker compose up -d`

Crear entorno Virtual: Se crea un entorno aislado para las dependencias del proyecto.

- **Comando:** `py -3.12 -m venv .venv`

NOTA: se puede utilizar cualquier versión de python 3.12+.

Activar entorno:

- **Comando:** `.\.venv\Scripts\Activate.ps1`

NOTA: Si PowerShell bloquea scripts ejecuta: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Instalar dependencias:

- **Comando:** `python -m pip install -r requirements.txt`

Ejecutar Backend:

- **Comando:** `uvicorn app.main:app --reload --app-dir src`



Para Linux / macOS:



Crear el archivo .env: Primero creamos el archivo para que la aplicación tenga su configuración y credenciales.

- **Comando:** `cp .env.example .env`

Verificación: Abre el archivo `.env` con `nano .env` o con `code .env` y asegúrate de que el usuario, contraseña, puerto y nombre de base de datos coincidan con el archivo `docker-compose.yml`.

Levantar Base de Datos (Docker): Esto inicia el contenedor de la base de datos.

- **Comando:** `docker compose up -d`

NOTA: A veces requiere `sudo docker compose up -d` si no tienes permisos configurados.

Crear entorno Virtual: Se crea un entorno aislado para las dependencias del proyecto.

- **Comando:** `python3 -m venv .venv`

Activar entorno:

- **Comando:** `source .venv/bin/activate`

Instalar dependencias:

- **Comando:** `pip install -r requirements.txt`

Ejecutar Backend:

- **Comando:** `uvicorn app.main:app --reload --app-dir src`

NOTA: En algunos sistemas puede ser necesario usar: `pip3 install -r requirements.txt`

Flujo con automatización

 **Para Windows (PowerShell):**

- **Comando:** `setup.bat`

 **Para Linux / macOS:**

- **Comando:** `./setup.sh`

Esto levanta la BD, crea el entorno virtual, instala dependencias y crea `.env` si no existe.

Ejecutar FastAPI manualmente:

- **Comando:** `uvicorn app.main:app --reload --app-dir src`

Con este proceso:

- Docker levanta la base de datos en un contenedor.
- Python ejecuta el backend.
- El backend utiliza las variables del archivo .env para conectarse a la base de datos.

Una vez ejecutado el comando para iniciar el backend: `uvicorn app.main:app --reload --app-dir src`

debería aparecer un mensaje similar a:

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [21048] using WatchFiles
INFO: Started server process [19532]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

- INFO: Uvicorn running on <http://127.0.0.1:8000>: Esto indica que el servidor FastAPI inició correctamente y está ejecutándose en el puerto 8000.

En caso de que todo funcione correctamente, ya será posible probar endpoints en las siguientes rutas:

- **Documentación interactiva (Swagger UI):** <http://127.0.0.1:8000/docs>
- **Documentación alternativa (ReDoc):** <http://127.0.0.1:8000/redoc>

3. Archivo docker-compose.yml

Este archivo define y configura los servicios necesarios para el proyecto utilizando Docker Compose.

En este caso, se configura un contenedor de PostgreSQL para la base de datos del sistema.

Explicación de la configuración:

- **version: "3.9"**
→ Define la versión de Docker Compose utilizada.
- **services**
→ Contiene los servicios o contenedores del proyecto.
- **postgres**
→ Servicio encargado de ejecutar la base de datos PostgreSQL.
- **image: postgres:16**
→ Utiliza la imagen oficial de PostgreSQL versión 16.
- **container_name**
→ Asigna un nombre personalizado al contenedor.



- **environment**

→ Define las variables de entorno:

- **POSTGRES_USER** → usuario de la base de datos.
- **POSTGRES_PASSWORD** → contraseña.
- **POSTGRES_DB** → nombre de la base de datos inicial.

- **ports**

→ Expone el puerto 5432 para permitir conexiones desde el host.

- **volumes**

→ Permiten persistencia y carga inicial de datos:

- **postgres_data**: almacena permanentemente la información de la base de datos.
- **arenasynctbv2.sql**: ejecuta automáticamente un script SQL de inicialización.

- **healthcheck**

→ Verifica periódicamente que PostgreSQL esté funcionando correctamente mediante `pg_isready`.

```
docker-compose.yml
1  version: "3.9"
2
3  services:
4    postgres:
5      image: postgres:16
6      container_name: arenasync-postgres
7      environment:
8        POSTGRES_USER: arenasync
9        POSTGRES_PASSWORD: arenasync
10       POSTGRES_DB: arenasynctbv
11      ports:
12        - "5432:5432"
13      volumes:
14        - postgres_data:/var/lib/postgresql/data
15        - ./arenasynctbv2.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
16      healthcheck:
17        test: ["CMD-SHELL", "pg_isready -U arenasync -d arenasynctbv"]
18        interval: 10s
19        timeout: 5s
20        retries: 5
21
22  volumes:
23    postgres_data:
24
```

NOTA: Al ejecutar `docker compose up -d`

Docker descargará la imagen de PostgreSQL, creará el contenedor **arenasync-postgres**, inicializará la base de datos **arenasynctbv** (creando una base de datos vacía) y ejecutará automáticamente el script SQL **arenasynctbv2.sql** (encargado de crear tablas, relaciones y datos iniciales del sistema).



4. La ejecución final de un **Hola Mundo** (SOLO BACKEND) que abarque:

Hola Mundo Backend con FastAPI + PostgreSQL

Objetivo

Realizar una prueba mínima del backend que permita:

- verificar la conexión con la base de datos,
- representar una entidad de la base de datos mediante una clase,
- probar el endpoint desde Postman.

Configuración de la conexión PostgreSQL:

Archivo [database.py](#): Este archivo crea la conexión entre FastAPI y PostgreSQL utilizando SQLAlchemy.

```
src > app > core > database.py > ...
1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import DeclarativeBase, sessionmaker
3
4  from app.core.config import settings
5
6  engine = create_engine(settings.database_url, pool_pre_ping=True)
7
8  SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
9
10
11 class Base(DeclarativeBase):
12     pass
13
14
15 def get_db():
16     db = SessionLocal()
17     try:
18         yield db
19     finally:
20         db.close()
21
```

- `engine = create_engine(...)` → Crea el motor que conecta a PostgreSQL usando la URL del `.env`.
- `SessionLocal = sessionmaker(...)` → Factoría para crear sesiones de BD (cada request obtiene una).
- `class Base(DeclarativeBase)` → Clase base que heredan los modelos (entidades) para que SQLAlchemy pueda mapearlos a tablas.
- `get_db()` → Función que inyecta la sesión en los endpoints (dependencia de FastAPI).

Representación de una entidad de la base de datos:



Archivo jugador.py: En este caso se representa la entidad Jugador mediante una clase de Python utilizando SQLAlchemy.

```
src > app > domain > models > jugador.py > ...
1  from sqlalchemy import Column, Date, Integer, Text
2
3  from app.core.database import Base
4
5
6  class Jugador(Base):
7      __tablename__ = "jugador"
8
9      id = Column(Integer, primary_key=True, autoincrement=False)
10     nombre_usuario = Column(Text, nullable=False)
11     correo_electronico = Column(Text, nullable=False)
12     contrasena_hash = Column(Text, nullable=False)
13     rol = Column(Text, nullable=False)
14     fecha_ultimo_acceso = Column(Date, nullable=False)
15     elo_global = Column(Integer, nullable=False)
16
```

Jugador es una clase Python que SQLAlchemy mapea directamente a la tabla jugador en PostgreSQL. Cada atributo con `Column()` se convierte en una columna de la tabla.

Creación de endpoints:

Archivo jugador_controller.py: Este archivo define los endpoints HTTP de la API para gestionar jugadores:

```
src > app > controllers > jugador_controller.py > ...
1  from fastapi import APIRouter, Depends, HTTPException, status
2  from sqlalchemy.orm import Session
3
4  from app.core.database import get_db
5  from app.domain.schemas.jugador import JugadorCreate, JugadorRead
6  from app.services.jugador_service import JugadorService
7
8  router = APIRouter(tags=["jugadores"])
9
10
11  @router.post("/jugadores", response_model=JugadorRead, status_code=status.HTTP_201_CREATED)
12  def crear_jugador(payload: JugadorCreate, db: Session = Depends(get_db)):
13      service = JugadorService(db)
14      return service.crear_jugador(payload)
15
16
17  @router.get("/jugadores/{jugador_id}", response_model=JugadorRead)
18  def obtener_jugador(jugador_id: int, db: Session = Depends(get_db)):
19      service = JugadorService(db)
20      jugador = service.obtener_jugador(jugador_id)
21      if jugador is None:
22          raise HTTPException(status_code=404, detail="Jugador no encontrado")
23      return jugador
24
```

- **POST /jugadores** → Crea un nuevo jugador



- Recibe datos en el body (JugadorCreate)
- Inyecta la sesión de BD automáticamente con Depends(get_db)
- Retorna el jugador creado (JugadorRead)
- **GET /jugadores/{jugador_id}** → Obtiene un jugador por ID
 - Recibe el ID como parámetro de ruta
 - Consulta la BD y retorna el jugador
 - Si no existe, lanza error 404

Prueba la conexión a la base de datos:

Archivo **health_controller.py**: Endpoint que prueba la conexión a BD.

```
src > app > controllers > health_controller.py > ...
1  from fastapi import APIRouter, Depends
2  from sqlalchemy import text
3  from sqlalchemy.orm import Session
4
5  from app.core.database import get_db
6
7  router = APIRouter(tags=["health"])
8
9
10 @router.get("/hola")
11 def hola_mundo(db: Session = Depends(get_db)):
12     db.execute(text("SELECT 1"))
13     return {"message": "Hola Mundo con BD"}
14
```

- **GET /hola** → ejecuta SELECT 1 en BD
- Si funciona → retorna {"message": "Hola Mundo con BD"}
- Si falla → error de conexión

Uso: Health check básico para verificar que la app y BD están conectadas.

Archivo principal FastAPI:

Archivo main.py: Es el archivo principal de FastAPI.

```
src > app > main.py > ...
1  from fastapi import FastAPI
2
3  from app.controllers.health_controller import router as health_router
4  from app.controllers.jugador_controller import router as jugador_router
5
6  app = FastAPI(title="ArenaSync Backend")
7
8  app.include_router(health_router)
9  app.include_router(jugador_router, prefix="/api")
10
11
12  @app.get("/")
13  def root():
14      return {"message": "ArenaSync Backend"}
15
```

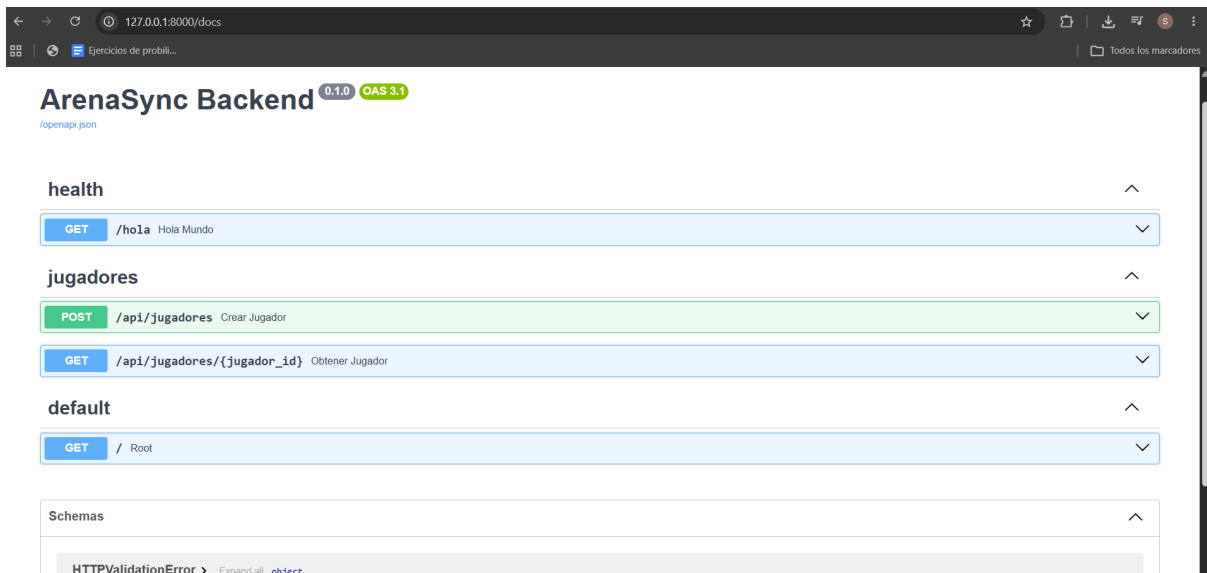
- `app = FastAPI(...)` → Crea la aplicación FastAPI
- `app.include_router(...)` → Registra los controladores (endpoints):
- `health_router` en / (chequeo de salud)
- `jugador_router` en /api (endpoints de jugadores)
- `@app.get("/")` → Endpoint raíz que retorna un mensaje

Ejecutar el backend: `uvicorn app.main:app --reload --app-dir src`

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [17384] using WatchFiles
INFO:      Started server process [13344]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Prueba desde el navegador:

- **Swagger UI:** <http://127.0.0.1:8000/docs>

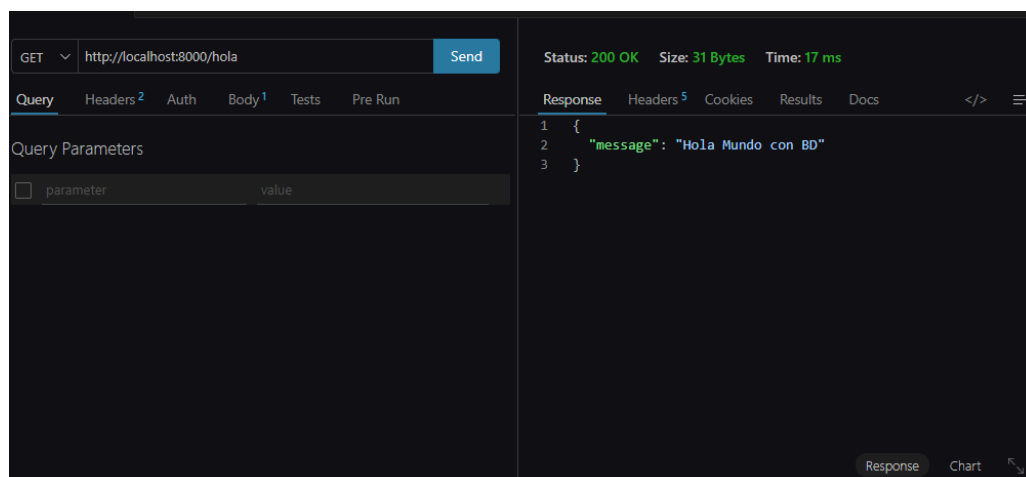


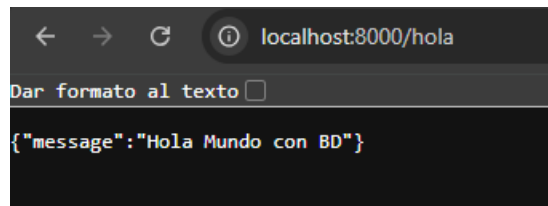
(Desde esta interfaz se pueden probar automáticamente los endpoints).

- **Pruebas desde Postman:** A continuación se detallan exactamente los pasos a seguir en Postman para verificar que el backend funciona.

GET 01 — Verificar que el servidor responde:

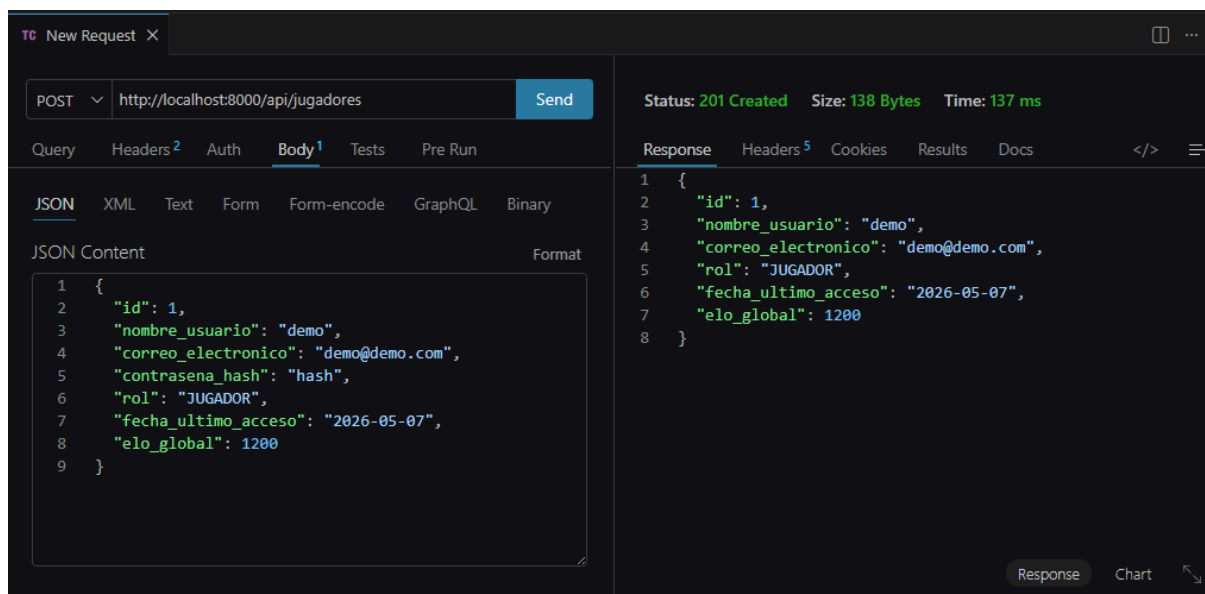
1. Abrir Postman y crear una nueva request.
2. Selecciona el método GET.
3. Escribir la URL: <http://localhost:8000/hola>
4. Hacer clic en Send.
5. Verificar que el Status sea 200 OK y el body muestre el mensaje.





POST 02 — Crear un usuario (instanciar la entidad):

1. Crear una nueva request en Postman.
2. Seleccionar el método POST.
3. Escribir la URL: <http://localhost:8000/api/jugadores>
4. Ir a la pestaña Body → seleccionar raw → tipo JSON.
5. Pegar un JSON con el cuerpo que se muestra abajo.
6. Hacer clic en Send.
7. Verificar que el Status sea 201 Created y la respuesta incluya el id=1.



GET 03 — Listar usuarios (leer la entidad de la BD):

1. Crear una nueva request en Postman.
2. Seleccionar el metodo GET.
3. Escribir la URL: <http://localhost:8000/api/jugadores/1>
4. Hacer clic en Send.
5. Verificar que la respuesta sea un array con el usuario creado en el paso 02.



GET

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "id": 1,
3   "nombre_usuario": "demo",
4   "correo_electronico": "demo@demo.com",
5   "contrasena_hash": "hash",
6   "rol": "JUGADOR",
7   "fecha_ultimo_acceso": "2026-05-07",
8   "elo_global": 1200
9 }
```

Status: 200 OK Size: 138 Bytes Time: 61 ms

Response Headers⁵ Cookies Results Docs

```
1 {
2   "id": 1,
3   "nombre_usuario": "demo",
4   "correo_electronico": "demo@demo.com",
5   "rol": "JUGADOR",
6   "fecha_ultimo_acceso": "2026-05-07",
7   "elo_global": 1200
8 }
```

← → ↻ localhost:8000/api/jugadores/1

☐ Dar formato al texto

```
{"id":1,"nombre_usuario":"demo","correo_electronico":"demo@demo.com","rol":"JUGADOR","fecha_ultimo_acceso":"2026-05-07","elo_global":1200}
```